



Loopify Juice

Customer Relationship Manager



A complete product documentation covering every feature of the CRM, the technology that powers it, and the three planned mobile apps that will extend the system: a delivery-partner app for riders, a customer-facing ordering app, and an AI chatbot for round-the-clock support.

Product	Loopify Juice CRM
Version	1.0.0
Stack	Node.js · Express · SQLite · JWT · vanilla JS frontend
Authoring	Single-team self-hosted deployment
Document type	Feature & architecture handbook

Cold-pressed juice deserves a cold-pressed CRM — fresh, lean, no preservatives.



01

Contents

- | | |
|----|--|
| 01 | Executive summary
What the CRM does, who it's for, and why this stack. |
| 02 | Architecture at a glance
Backend, database, frontend, and how they fit together. |
| 03 | Setup & first run
From zero to logged in, in under a minute. |
| 04 | Dashboard
The home screen — every KPI explained. |
| 05 | Customers
The single source of truth for every juice drinker. |
| 06 | Orders & delivery tracking
From cart to doorstep, with a live timeline. |
| 07 | Subscriptions
Recurring revenue, on autopilot. |
| 08 | Products & inventory
The juice catalogue with live stock movements. |
| 09 | Leads & follow-ups
A Kanban pipeline from cold lead to paying customer. |
| 10 | Delivery partners
Managing the riders who close the last mile. |
| 11 | WhatsApp inbox
Real webhook, two-way conversation, lead conversion. |
| 12 | Calendar
One view for orders, deliveries, and follow-ups. |
| 13 | Reports
Revenue, AOV, conversion, and product performance. |
| 14 | Settings & data
Business info, theme, backup, restore, reset. |
| 15 | Security & auth
JWT, bcrypt, and what we leave to you. |
| 16 | Future: Delivery Partner App
How riders will get their own mobile app. |
| 17 | Future: Customer App
How customers will order without WhatsApp. |



18

Future: AI Chatbot

How a smarter bot will handle the front line.

19

API reference

Every endpoint, in one place.

20

Roadmap & extensions

What's next, and how to add it.



02

Executive summary

Loopify Juice CRM is a complete operating system for a cold-pressed juice business. It tracks every customer, every order, every subscription, every juice bottle in stock, every rider on the road, and every WhatsApp conversation — in one place, with a single sign-in.

It's **self-hosted**: the entire system runs on a single Node.js process you launch with a double-click. The database is a single SQLite file you can back up with a copy-paste. There are no external services to subscribe to, no monthly SaaS fee, and no vendor lock-in. The default install runs comfortably on a \$5/month VPS.

Despite the lean stack, the feature surface is full: dashboard, customers, orders with live delivery tracking, recurring subscriptions, products with auto-decrementing inventory, a Kanban leads pipeline with drag-and-drop, a WhatsApp inbox wired to a real webhook (Meta Cloud and Twilio formats supported), a calendar view, reports, printable invoices, dark mode, command palette, JSON/CSV exports — and a default admin login that's ready in 60 seconds.

Who it's for

A juice brand running between roughly 30 and 3,000 orders a day across one or several cities. Owners and operations managers will use the CRM directly; the planned mobile companion apps (documented later in this PDF) will extend it to riders and end customers.

Why this stack

We optimized for three things, in this order: **(1) operator empowerment** — the system runs entirely on the operator's hardware, no vendor between you and your data; **(2) speed of iteration** — no build step, no compile, no transpile, edit a file and refresh; **(3) cost** — SQLite + Node fits inside the smallest VPS or even a Raspberry Pi.



03

Architecture at a glance

The CRM is a classic three-tier app: a SQLite database underneath, an Express REST API in the middle, and a vanilla-JS single-page frontend on top. Everything lives in one repository.

Folder structure

```
Juice CRM/  
■■■ start.bat / start.sh one-click launchers  
■■■ README.md  
■■■ backend/  
■ ■■■ server.js Express entry  
■ ■■■ db.js SQLite schema + helpers  
■ ■■■ auth.js JWT + bcrypt  
■ ■■■ seed.js / reset.js  
■ ■■■ routes/ 11 REST route files  
■ ■■■ data/loopify.db auto-created SQLite file  
■■■ frontend/  
■■■ index.html, login.html  
■■■ css/styles.css  
■■■ js/  
■■■ icons, utils, state, api, auth, ui, router, app  
■■■ pages/ (11 page modules)
```

How a request flows

When you click **Add customer** in the UI, the frontend's `api.js` sends `POST /api/customers` with a JWT in the *Authorization* header. Express receives it, the `requireAuth` middleware verifies the token via `jsonwebtoken`, then the route handler inserts a row using a prepared `better-sqlite3` statement and returns the new record as JSON. The frontend updates `State.customers` and re-renders the page. Total round trip: under 20 ms on a laptop.

Data persistence

Everything lives in a single SQLite file at `backend/data/loopify.db`. WAL journal mode is enabled for safe concurrent reads. Foreign keys cascade so deleting a customer automatically removes their orders and subscriptions. Backup is literally a file copy.



04

Setup & first run

The CRM is designed to run with zero configuration. Here's the entire setup procedure:

1. Install Node.js v18 or newer from nodejs.org (one-time, ~2 minutes).
2. Open the *Juice CRM* folder.
3. Double-click `start.bat` (Windows) or run `./start.sh` (macOS/Linux).
4. The launcher installs dependencies on first run (~30 seconds), starts the server, and opens `http://localhost:4000` in your browser.
5. Sign in with the default admin credentials displayed in the terminal.

Default login

Email	admin@loopifyjuice.in
Password	loopify@2026
Change in	Settings → Account → Change password

Production hardening

Before exposing the CRM on a public URL, set a strong `JWT_SECRET` in `backend/.env` (any long random string), change the admin password, and put `nginx` in front for `HTTPS`. Total time: 10 minutes.



05

Dashboard

The dashboard is the page that loads when you sign in. It's designed to answer one question in under three seconds: *is the business healthy today?*

The six KPIs

Total revenue	Sum of all delivered orders, ever. Shows growth trajectory vs. previous 30 days.
Revenue (30d)	Last 30 days only. The number you watch week to week.
Customers	All customers, plus how many joined this month.
Active subscriptions	Subscriptions currently in 'active' status — your recurring base.
Pending deliveries	Orders currently in confirmed, preparing, or out-for-delivery state.
Low-stock items	Products at or below their low-stock threshold. Click to refill.

Charts

Beneath the KPIs are a 30-day revenue line chart (delivered orders only) and an order-status doughnut showing how your pipeline is distributed. Both are rendered live with Chart.js using the active theme colours.

Today's three lists

Three side-by-side cards show what needs your attention *today*: deliveries currently out for delivery, leads with a follow-up due today, and products that have crossed the low-stock threshold. Each item is clickable and deep-links to the relevant detail view.

Recent activity

The activity feed at the bottom is an append-only log of operational events: customer additions, order status changes, subscription edits, lead stage transitions, WhatsApp messages received. It's a quick scrubbing tool for 'what happened?' questions.



06

Customers

The Customers page is your single source of truth for every person who has ever bought a Loopify juice. Click any row to open the detail drawer.

What we track per customer

Identity	Full name, phone, email.
Location	Area / locality (used for delivery zoning) and full address.
Tags	Free-form labels like vip, vegan, weekly, office, bulk — used for segmentation.
Notes	Allergies, preferences, anything to remember.
Computed	Lifetime value, total orders, average order value, last order date.
Relations	Linked order history and subscription history.

Search, sort, paginate

The table is sortable on every column (name, phone, area, orders, lifetime value, last order date), paginates at 25 rows, and the global search box in the topbar matches across name, phone, email, and area. Searching across 1,000 customers is instant thanks to the in-memory filter.

Import & export

Export CSV dumps every customer plus their computed lifetime value to a spreadsheet-friendly CSV. **Import CSV** (in the standalone version) accepts the same format and adds new records. For full data migration, use the JSON backup on the Settings page — it preserves every relationship.

Tip — Lead conversion. When you convert a lead to a customer (from the Leads page or the WhatsApp inbox), the system tags them automatically with `from-lead` plus the lead source, so you can later filter to see which campaigns produced the highest-LTV customers.



07

Orders & delivery tracking

Orders is the heaviest module in the CRM. Every order moves through six possible statuses, each transition is logged with a timestamp, and the system auto-decrements product stock the moment an order is marked delivered.

Status lifecycle

Stage	Meaning	Typical trigger
Pending	Order received, not yet confirmed	Customer placed order, awaiting acknowledgement
Confirmed	Order acknowledged	Operator confirms ingredients & slot
Preparing	Juice is being cold-pressed	Production starts
Out for delivery	Rider picked up the bottles	Partner assigned & on the way
Delivered	Customer received the order	Stock auto-decrements, revenue counted
Cancelled	Order will not be fulfilled	Customer cancellation or stockout

Order detail view

Clicking any order opens a two-column drawer. Left side: customer card, assigned delivery partner (changeable inline via dropdown), and a fully itemised receipt with subtotal, delivery fee, and final total. Right side: the live tracking timeline showing each stage's timestamp, a row of *Update status* buttons for one-click progression, an **Edit order** button, a **Print invoice** button (opens a print-ready window in your browser), and a **WhatsApp customer** shortcut that pre-fills a status update message.

Auto-stock decrement

When an order transitions *into* the **Delivered** state for the first time, the backend automatically deducts each line item's quantity from the matching product's stock and writes a row to the `stock_moves` audit table. This is implemented as a single SQL transaction, so an interrupted delivery operation can't corrupt your inventory.

Filtering & exports

Tabs at the top of the Orders page filter by status. Every tab shows a live count. The **Export CSV** button downloads the filtered set with all line items flattened — ready for accounting, Excel pivot tables, or external analytics.



08

Subscriptions

Subscriptions turn one-off customers into a recurring revenue base. Each subscription is a stored plan with a customer, a frequency, a set of line items, and a next-delivery date. It's the system's way of saying 'on this date, this customer should receive these bottles'.

Frequencies supported

Daily	Used for shots and morning juices. Counts as 30× per month for MRR.
Weekly	Wellness packs. 4× per month.
Biweekly	Family-pack deliveries. 2× per month.
Monthly	Detox plans, bulk orders. 1× per month.

MRR calculation

The Subscriptions page shows an estimated Monthly Recurring Revenue: for each active subscription, multiply the line-item total by the monthly multiplier above, and sum across all active plans. This gives you a forecast that updates the moment you add or pause a subscription.

Pause & resume

Subscriptions have three states: **active**, **paused** (holidays, illness, or a delivery freeze), and **cancelled**. Paused subscriptions stop counting toward MRR but remain visible and one click away from being resumed. Inline toggle buttons on the list view make this a two-second operation.



09 Products & inventory

The Products page is your juice catalogue and live inventory ledger in one view. Each card shows price, current stock, and a coloured progress bar that turns red when stock dips at or below the low-stock threshold.

Per-product fields

Name	Product display name.
Emoji	A single emoji used as the visual identifier (e.g. 🟢 for Green Detox).
Category	Free-form grouping: Detox, Energy, Wellness, Refresher, Shot, etc.
Price (₹)	Selling price in rupees.
Stock	Current units available. Decrements automatically on delivery.
Low-stock threshold	When stock falls to this number or below, the product appears in low-stock alerts.
Description	Ingredients list, free text.

Stock movements log

Every change to a product's stock count writes a row to the `stock_moves` table with the delta, the reason, and a timestamp. Opening any product shows the last five movements at the bottom of the edit form. Sources of movement: manual stock adjustments (operator changed the number directly), order deliveries (auto-decrement when an order hits delivered).

Tip — Stock value. The page header shows the total rupee value of stock on hand (sum of *stock* × *price* across every product). It's a quick eyeball check before reordering ingredients.



10

Leads & follow-ups

Leads is the sales pipeline. It's organised as a five-column Kanban board, and you progress a lead from left to right by dragging the card or by editing the stage in the lead form.

The five stages

New	Just walked in, just messaged on WhatsApp, just filled the website form.
Contacted	We've responded but no buying signal yet.
Qualified	Real intent. Pricing discussed. Ready to convert.
Converted	Became a customer. Tracked here for source attribution.
Lost	Went silent or chose a competitor.

Drag-and-drop

The Kanban is built on the native HTML5 Drag & Drop API. Pick up a card and drop it on any column to instantly persist the new stage to the database and update the activity log. No save button, no confirmation dialog — the motion is the commit.

Follow-up dates

Every lead can carry a follow-up date. The dashboard surfaces today's follow-ups, the page header counts overdue follow-ups in red, and the notification bell pings when due dates arrive. The point is to make 'I forgot to call Riya back' impossible.

One-click conversion

The **Convert to customer** button on any lead form creates a real customer record carrying the lead's name, phone, source (stored as a tag), and notes, then updates the lead to **Converted**. The conversion is logged in activity so you can audit which leads converted and when.



11

Delivery partners

The Delivery Partners page is the directory of every rider, driver, and walker your business uses. Each partner card shows their name, phone, area of operation, vehicle, active status, and three live counters: currently active orders, lifetime delivered, and total ever assigned.

Why this matters

Without partner tracking, the question 'who has Neha's order right now?' is impossible to answer quickly. With it, every order carries a partner reference, the order detail view shows the partner inline, and the partner detail view lists every order they've ever been responsible for — sorted by date with their statuses.

Inline reassignment

If a partner calls in sick or the rider rotation changes, every order detail page has a **Reassign** dropdown that lists every currently-active partner. Pick a new name, the assignment persists instantly, and the next time anyone opens that order they see the new partner without needing to refresh anything.

Coming next: the planned *Delivery Partner mobile app* (covered in detail later in this document) will give each rider a personal screen showing their queue, with one-tap status updates and turn-by-turn navigation links.



WhatsApp inbox

WhatsApp is where most Indian juice customers actually live. The CRM treats it as a first-class inbox alongside the orders system. Inbound messages stream in via a real webhook; outbound messages can go through Twilio's API when configured.

The webhook

The backend exposes `POST /api/whatsapp/webhook` as a public endpoint. It auto-detects payloads from **Meta Cloud API** (Facebook's official WhatsApp Business platform) and **Twilio** (the most popular reseller), plus a generic JSON format `{phone, name, text}` useful for custom bots. The handshake `GET /api/whatsapp/webhook?hub.verify_token=...` returns the challenge token when the `WHATSAPP_VERIFY_TOKEN` env variable matches, which is what Meta/Twilio need to confirm your URL.

Inbox UI

The inbox renders like a familiar WhatsApp client: a left rail of conversations sorted by last activity, an unread badge on the sidebar, and a right pane with the full conversation in chat bubbles. Each thread is automatically matched to a customer record by phone number; if no match exists, you get one-click buttons to **Save contact** (creates a customer) or **Create lead** (creates a lead pre-populated with their first message as notes). For existing customers, the **New order** button is right there in the conversation header.

Outbound via Twilio

When you type a reply and hit Send, the backend stores the message locally *and* (if Twilio is configured) actually sends it through Twilio's WhatsApp REST API. Configure by adding three lines to `backend/.env`:

```
TWILIO_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_TOKEN=your_auth_token
TWILIO_FROM=whatsapp:+14155238886
```

Without these, the message is still recorded in the inbox for your records, and the response includes `provider.sent: false, reason: 'Twilio not configured'` so you can tell at a glance.

Quick-reply templates

The compose bar has a row of one-click templates for the most common replies: 'Our weekly wellness pack starts at ■999', 'Your order is being prepared', 'Cold-pressed and preservative-free', and so on. Clicking a template loads it into the input for you to tweak before sending.



13

Calendar

The Calendar consolidates three different time-bound entities into one month view: orders created on a given day (green pills), subscription deliveries scheduled for that day (purple pills), and lead follow-ups due that day (amber pills). Today is highlighted in a soft cream tint with a green border.

Each pill is clickable and opens the relevant detail drawer. Months with more than three events per day show a *+N more* indicator. Navigation: previous-month, next-month, and a **Today** button to jump back to the current month.



14

Reports

The Reports page is for the question 'how is the business doing over time?'. A range selector at the top picks the window (7, 30, 90, or 180 days), and every metric on the page recomputes against that window.

Headline metrics

Revenue	Delivered orders in the selected window.
Average order value (AOV)	Revenue ÷ number of delivered orders.
Orders	Total orders created in window, with a count of cancellations.
Lead → customer conversion %	What share of leads ever convert to customers.

Breakdowns

A bar chart of daily revenue over the window. A horizontal bar chart of revenue by customer area (Indiranagar vs. Koramangala etc.). A sortable list of top customers by spend with their order count and primary area. A sortable list of products by units sold and revenue. Each row in the customer list deep-links to the customer detail drawer; product names link to the product editor.



15

Settings & data management

Settings is the back office. It's organised into three cards: Business information, Account & appearance, and Data management.

Business information

Name, address, phone, email, and optional GSTIN. This is what appears on printed invoices. Saved as a single JSON blob in the `settings` table so it survives backup/restore round-trips.

Account & appearance

Change password — requires the current password (defends against accidental terminal access).

Dark mode toggle — swaps the Botanical Garden palette to its dark variant; preference is stored in `localStorage` and applied immediately. **Sign out** — clears the JWT and redirects to login.

Data management

Download backup (JSON) exports every row of every table to a single JSON file. Open it in any text editor to inspect or grep. **Restore from backup** takes the same JSON file and replaces the live database with it (the action requires confirmation). **Customers (CSV)** and **Orders (CSV)** dump those tables in spreadsheet-friendly format. **Reset to demo data** wipes everything and reloads the sample dataset — useful for training new staff or rehearsing a flow.



Security & authentication

Authentication uses industry-standard JWT (JSON Web Tokens) signed with HMAC-SHA256 and a secret you control. Passwords are hashed with bcrypt (cost factor 10). Tokens last 30 days by default — short enough to limit blast radius if a laptop is lost, long enough to not be annoying.

What we secure

- Every `/api/*` route requires a valid JWT, except the WhatsApp webhook (which is intentionally public — that's the whole point of webhooks) and the login endpoint.
- Passwords are never stored in plain text. Even an attacker with the SQLite file cannot read them.
- CORS is enabled with permissive defaults for local development; tighten it for production by setting an origin allowlist.

What you should harden

- Change `JWT_SECRET` from the default to a long random string before going public.
- Change the default admin password from `loopify@2026` the moment you sign in.
- Put HTTPS in front using nginx, Caddy, or Cloudflare Tunnel.
- Set a rate limiter in front of `/api/auth/login` for the open internet.



17

Future: the Delivery Partner App

The CRM today lets *you* assign and track partners. The planned Delivery Partner app gives each *rider* their own screen with their own queue, on their own phone — no calls, no SMS chains, no 'where is my order?' messages.

What the rider sees

Today's queue. A vertically stacked list of every order assigned to that rider for today, in delivery order. Each card shows the customer's name, area, address, phone (one-tap to call), an estimated route position, and the order total (in case it's a Cash-on-Delivery run).

Active delivery. Tapping a card expands it to show the bottle list, special notes, and three big buttons: *Started* (moves the order to out-for-delivery), *Delivered* (moves to delivered, captures GPS coordinates), and *Customer not reachable* (creates a follow-up task back in the CRM).

Navigation. A single button opens Google Maps or Apple Maps with the customer's address pre-filled.

Performance. A profile screen showing today's delivered count, on-time rate, and customer rating (when added).

How it'll talk to the CRM

The app will be a thin React Native client over the existing REST API. We'll add three partner-scoped endpoints: `GET /api/partner/me/queue` (today's orders), `PATCH /api/partner/orders/:id/status` (status update with optional GPS), and `POST /api/partner/auth/otp` (login via phone-OTP rather than email/password, since riders rarely have email accounts).

Authentication

Partners will log in with their phone number plus a 6-digit OTP sent over WhatsApp (via the same Twilio integration used for the inbox). Token scope is partner-only — they can read and update their own orders, nothing else. They cannot see customer LTV, revenue numbers, or other riders' queues.

Offline-first

Riders move in and out of mobile signal all day. The app will queue status updates locally and sync when connectivity returns. The local store will use IndexedDB on iOS/Android via WatermelonDB or RxDB. The CRM remains the source of truth.



18

Future: the Customer App

Right now the buying journey ends in WhatsApp. The planned Customer App will give your end customers a real ordering interface — without forcing them to abandon WhatsApp where they already trust you.

What the customer sees

Storefront. Browse the juice catalogue by category (Detox, Energy, Wellness, Refresher, Shots). Each product card shows the photo, ingredients, price, and a quick-add button.

Cart & checkout. Standard cart with quantity steppers. Delivery address picker (saved from previous orders or new entry). Slot picker (morning, afternoon, evening). Razorpay or Cashfree integration for online payments; Cash-on-Delivery is also supported.

Subscriptions. A one-tap upgrade from any cart to 'deliver this every week' — same plan structure the CRM already understands.

Order tracking. The same status timeline the operator sees inside the CRM is mirrored to the customer's app, with push notifications at each transition.

Account. Order history, manage subscriptions (pause, skip, change frequency), update address, reorder favourites in one tap.

How it'll talk to the CRM

Like the partner app, the customer app will be React Native over the existing REST API, with five new customer-scoped endpoints: `POST /api/customer/auth` (phone OTP signup/login), `GET /api/customer/products` (public catalogue), `POST /api/customer/orders` (place order), `GET /api/customer/me/orders` (their order history), and `POST /api/customer/subscriptions` (manage their own plans).

Why a customer app at all

WhatsApp ordering is great until your business hits ~50 orders/day. After that, every order in WhatsApp is a manual data-entry job. An ordering app pushes the data-entry cost back to the customer (who is happy to do it for their own order) and frees your team to focus on production, delivery, and customer relationships rather than typing.



Future: the AI Chatbot

The WhatsApp inbox today is operator-driven — a human reads each message and replies. The planned AI chatbot puts an intelligent first-responder in front of that inbox, handling the easy 70% of conversations automatically and escalating the hard 30% to a human.

What it'll handle without human help

- **Catalogue queries.** 'What juices do you have?' → fetch products, reply with the full menu and prices.
- **Order status.** 'When is my order coming?' → look up the customer's most recent order by phone number, reply with the live status from the tracking timeline.
- **Delivery area.** 'Do you deliver to HSR Layout?' → check the customer's area against the active delivery zones.
- **Subscription enquiry.** 'How much for a weekly plan?' → quote the standard plans with prices.
- **FAQ.** 'Are these preservative-free?' / 'How long is shelf life?' / 'Can I gift it?' — answered from a curated FAQ.
- **Lead capture.** Any first-time number is automatically registered as a new lead with the chat history as the notes field.

How it'll work technically

The webhook will route every inbound WhatsApp message through three layers before deciding what to do:

- 1. Intent classifier.** A small LLM (Claude Haiku or GPT-4o-mini works at sub-cent cost per message) classifies the message into an intent: status, catalogue, area, subscription, faq, complaint, other.
- 2. Data lookup.** The CRM's existing endpoints provide the data the bot needs to compose a personalised reply. Example: for a status query, the bot pulls the most recent order by phone, then templates the live tracking status into the response.
- 3. Response composer.** A short prompt to the LLM with the looked-up data + the customer's actual question + brand voice guidelines produces the reply. Reply is sent via the existing Twilio integration.

When it'll hand off to a human

Three triggers will escalate from bot to human: (1) explicit complaint or refund request detected by the classifier; (2) two consecutive turns where the bot can't answer confidently; (3) a message containing keywords like 'manager', 'speak to someone', or 'urgent'. On escalation, the conversation flips to a banner that says 'Bot paused — operator handling' and the unread badge fires in the CRM sidebar.

Cost & latency

At Haiku/4o-mini pricing, automated replies cost roughly ■0.04–■0.10 per conversation. End-to-end latency target: under 4 seconds from inbound message to outbound reply, dominated by the WhatsApp delivery round-trip, not the LLM.



API reference

All endpoints under `/api` require a JWT bearer token in the *Authorization* header, except `/api/auth/login` and `/api/whatsapp/webhook`.

Auth

Method	Path	Purpose
POST	<code>/api/auth/login</code>	Email + password → JWT
GET	<code>/api/auth/me</code>	Current user info
POST	<code>/api/auth/change-password</code>	Change password (auth)

Customers

Method	Path	Purpose
GET	<code>/api/customers</code>	List all customers
GET	<code>/api/customers/:id</code>	One customer + orders + subs + LTV
POST	<code>/api/customers</code>	Create
PUT	<code>/api/customers/:id</code>	Update
DELETE	<code>/api/customers/:id</code>	Delete (cascades to orders & subs)

Orders

Method	Path	Purpose
GET	<code>/api/orders</code>	List (optional <code>?status=</code> , <code>?customerId=</code> , <code>?partnerId=</code> , <code>?limit=</code>)
GET	<code>/api/orders/:id</code>	One order
POST	<code>/api/orders</code>	Create
PUT	<code>/api/orders/:id</code>	Update
PATCH	<code>/api/orders/:id/status</code>	Change status (+ auto stock decrement on delivered)
PATCH	<code>/api/orders/:id/partner</code>	Reassign partner
DELETE	<code>/api/orders/:id</code>	Delete



Subscriptions, Products, Partners, Leads

Method	Path	Purpose
GET/POST/PUT/DELETE	/api/subscriptions	Standard CRUD
PATCH	/api/subscriptions/:id/toggle	Active ↔ paused
GET/POST/PUT/DELETE	/api/products	Standard CRUD
GET/POST/PUT/DELETE	/api/partners	Standard CRUD
GET/POST/PUT/DELETE	/api/leads	Standard CRUD
PATCH	/api/leads/:id/stage	Move stage
POST	/api/leads/:id/convert	Convert to customer

WhatsApp

Method	Path	Purpose
GET	/api/whatsapp/webhook	Public verification handshake
POST	/api/whatsapp/webhook	Public inbound (Meta/Twilio/generic)
GET	/api/whatsapp/threads	All threads with last message + unread
GET	/api/whatsapp/messages?phone=...	Full thread
POST	/api/whatsapp/reply	Send (via Twilio if configured)
POST	/api/whatsapp/mark-read	Clear unread for a thread
POST	/api/whatsapp/simulate	Inject demo message (dev only)
GET	/api/whatsapp/unread-count	For sidebar badge

Reports & Settings

Method	Path	Purpose
GET	/api/reports/dashboard	All dashboard data in one shot
GET	/api/reports/summary?range=N	Reports page data for last N days
GET	/api/reports/activity	Recent activity feed
GET	/api/settings	All key/value settings
PUT	/api/settings/:key	Update one setting
GET	/api/settings/backup/download	Download full DB as JSON
POST	/api/settings/backup/restore	Restore from JSON (wipes first)
POST	/api/settings/reset	Wipe + reseed demo data



Roadmap & extensions

The current CRM is feature-complete for the core operating loop of a small juice business. Future work falls into three buckets:

Near-term polish

- Audit log surfacing — show who changed what, when (the data already exists in the activity table).
- Bulk operations — select multiple customers/orders for delete, tag, or status change.
- Email integration — order confirmations and invoices via SendGrid or Mailgun.
- Map view — orders pinned to a city map with partner positions.
- Multi-user accounts — operations staff with role-based scopes (admin, ops, accountant).

The three apps

Detailed in sections 17, 18, and 19. Each is a React Native client over the existing REST API plus a small number of new scoped endpoints. Estimated build time: 3–4 weeks each for a v1 ready to ship.

Operational tooling

- Recurring scheduled jobs — auto-generate tomorrow's delivery list from active subscriptions.
- Loyalty points & referrals — track points per ■100 spent, referral codes, redemption.
- Invoicing integrations — sync to Zoho Books or Tally for accounting.
- POS integration — when you open a physical storefront, the same product catalogue and customer database powers the till.



Thank you

This document describes the CRM as of the v1.0.0 release. The system is designed to be edited — every file is human-readable, the database is portable, and the architecture is intentionally simple. If you can read this, you can change it.

